1. Threading & Concurrency

`Runnable` vs. `Thread` Class

In Java, you can define and start a thread using either the `Thread` class or the `Runnable` interface. While both achieve multithreading, their design use cases differ significantly.

- **`Runnable` Interface (Preferred):** Represents the *task* to be executed. It separates the job from the execution mechanism.
- **`Thread` Class:** Represents the *actual worker thread* doing the execution.

Feature	Runnable Interface	Thread Class
Inheritance	Allows inheriting other classes (Java supports multiple interface implementation).	Restricts inheritance because Java does not support multiple class inheritance.
Object Sharing	Multiple threads can share the same `Runnable` instance, making it ideal for shared-state tasks.	Each thread requires a unique instance of a class extending `Thread`.
Design Paradigm	Promotes loose coupling (separation of task and runner).	Promotes tight coupling (task and runner are bound together).

The Executor Framework

Manually creating `new Thread()` for every task introduces high overhead (thread creation, context switching, and destruction costs). The **Executor Framework** (`java.util.concurrent`) solves this by decoupling task submission from execution using thread pools.

Java
```java
// Example: Using a fixed thread pool instead of manual Thread creation
ExecutorService executor = Executors.newFixedThreadPool(3);

for (int i = 0; i < 10; i++) {
    executor.execute(() -> System.out.println("Executing task on: " + Thread.currentThread().getName()));
}
executor.shutdown();
```

- **Core Component:** `ThreadPoolExecutor` manages a pool of worker threads and a work queue.
- **Workflow:** When a task is submitted, the framework tries to assign it to an idle core thread. If all core threads are busy, the task is placed in a blocking queue. If the queue fills up, new threads are created up to the maximum pool size.

2. Collections Framework: HashMap Internals

Internal Working & Collision Management

A `HashMap` operates on the principle of **Hashing**. Internally, it uses an array of buckets (nodes) to store key-value pairs.

1. **Putting a Value:** When you call `put(K key, V value)`, Java calculates the key's hash using `hash(key)`. It then maps this hash to a specific index in the bucket array using the formula:

   Index = hash mod n

   (where n is the array length).

2. **Handling Collisions (Chaining to Treeifying):** * **Linked List Chaining:** If two distinct keys map to the same bucket index (a collision), they are stored as a singly linked list at that index.
   - **Treeification (Java 8+):** If a single bucket's linked list grows past a **Threshold of 8** elements, and the total map capacity is at least 64, the linked list is converted into a **Balanced Red-Black Tree**. This improves the worst-case search time from O(n) to O(log n).

The `hashCode()` and `equals()` Contract

To successfully retrieve an object from a `HashMap`, you must override both methods together.

- **The Contract:** If two objects are equal according to `equals(Object)`, they **must** return the same integer result from `hashCode()`.
- **Why they must be overridden together:**
  - If you override `equals()` but not `hashCode()`, two logically "equal" objects will generate different bucket indexes. The `HashMap` will look in the wrong bucket during a `get()` call and fail to find your object.
  - If you override `hashCode()` but not `equals()`, different objects might land in the same bucket, but the map won't be able to distinguish them, either overwriting the wrong data or failing to retrieve it.

3. Core Java Concepts

String Immutability

Once a `String` object is created in Java, its value cannot be modified. Any operation that appears to alter a string (like `.concat()` or `.toUpperCase()`) actually creates a brand-new `String` object in memory.

- **Why?** Security (strings are used for file paths, network URLs, and database credentials), Thread-safety (immutable objects are inherently thread-safe), and the Optimization of the **String Constant Pool** (multiple references can point to the exact same literal to save heap space).

The `volatile` Keyword

The `volatile` keyword ensures that changes made to a variable by one thread are immediately visible to all other threads.

- **The Problem:** To maximize performance, CPU cores cache variables in local registers/caches rather than reading directly from main memory.
- **The Volatile Fix:** It forces the JVM to read and write the variable directly from and to **Main Memory**. It guarantees **visibility**, but it *does not* guarantee **atomicity** (e.g., `count++` is still not safe on a volatile variable because it involves three separate operations: read, modify, write).

Serializable

`Serializable` is a marker interface (an interface with no methods). It flags to the JVM that an object's state can be flattened into a byte stream.

- **Purpose:** Used for saving an object's state to a file, database, or transmitting it over a network to another JVM.
- **Note:** Fields marked with the `transient` keyword will *not* be serialized; they revert to their default values (e.g., `null` or `0`) upon deserialization.

4. Modern Java: The `Optional<T>` Class

Introduced in Java 8, `Optional<T>` is a single-value container that either contains a non-null value or is empty. It is designed to explicitly represent the absence of a value, replacing the risky practice of returning `null`.

Advantages Over Raw Null Checks

- **Defensive API Design:** It forces the calling developer to explicitly handle the case where a value might be missing, drastically reducing `NullPointerException` risks.
- **Readability:** Replaces nested, clunky `if (obj != null)` blocks with clean, functional fluent chains.

Practical Use Comparison

Old Way (Prone to NullPointerExceptions):

Java

```
// Risk of NullPointerException at every step if street or address is null
String streetName = user.getAddress().getStreet().getName();
```

Modern Way (Using Optional):

Java

```java
String streetName = Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getStreet)
    .map(Street::getName)
    .orElse("Unknown Street"); // Clean fallback mechanism
```